School of Science and Engineering

Data Ethics and Research Methods

Coursework 2: Research Project Proposal

Design and evaluation of an Agentic AI Study Planner for Managing University Coursework and Deadlines

Author:
Eberenna Peter Nwachukwu

Student Number:
S2550239

Abstract

Agentic AI systems are characterised by autonomous planning, tool use, and iterative reasoning capabilities. They represent an important area of research in artificial intelligence. This proposal outlines a research project to design and evaluate a prototypical agentic AI study planner system. The project addresses the technical challenge of creating an autonomous agent capable of dynamic task scheduling under complex constraints, using study planning as a motivating use case to demonstrate and evaluate core agentic capabilities. The primary aim is to develop a functional prototype that showcases agentic AI planning and scheduling mechanisms and to conduct an an experimental evaluation of alternative planning approaches or agent frameworks. The system will be designed to autonomously generate feasible weekly schedules, handle deadline constraints, manage time allocations, and adapt to changing requirements through iterative planning cycles. The research uses a design-and-implementation methodology. It begins with an architectural specification of an agentic planner, followed by prototype development using appropriate frameworks (e.g., LLM-based agent toolkits or custom planning algorithms) and systematic experimental evaluation. The evaluation will assess the feasibility of the schedule, robustness to changes, computational efficiency, and comparative performance with baseline scheduling methods. This provides a comprehensive assessment of the system's capabilities. Expected contributions include insights into the practical feasibility of agent-based AI for constraint-based scheduling, identification of limitations in current planning approaches, and empirical comparison of different agent architectures or planning strategies. The work provides a foundation for understanding how autonomous agents can handle structured, multi-constraint planning problems.
Keywords: Agentic AI, autonomous agents, study planning, task scheduling, LLM-based agents, planning algorithms, prototype evaluation, constraint-based scheduling

"Except where explicitly stated all work in this document is my own"

Name: Eberenna Peter Nwachukwu

Date: December 3, 2025

Contents

1 Introduction **4**
1.1 Project Background 4
1.2 Project Motivation 4
1.3 Research Questions and Hypothesis 5

2 Description of the Project **5**
2.1 Problem Statement 5
2.2 Project Goal and Objectives 6
2.3 Project Deliverables and Milestones 6
2.4 Project Roadmap 7

3 Literature Review **7**
3.1 Agentic AI and Autonomous Agents 7
3.2 Planning and Scheduling Algorithms for Intelligent Agents 8
3.3 LLM-Based and Tool-Using Agent Frameworks 8
3.4 Intelligent Scheduling and Study Planning Systems 9
3.4.1 Critical Gaps Identified 9

4 Project Methods **10**
4.1 Project Type 10
4.2 Requirements Elicitation and Definition 10
4.2.1 Scenario Definitions: 10
4.2.2 Functional Requirements: 11
4.2.3 Non-Functional Requirements: 11
4.3 Design: Agent Architecture 11
4.3.1 Perception Module: 11
4.3.2 Internal Representation: 11
4.3.3 Planning Module (core Component: 12
4.3.4 Action/Output Module: 12
4.3.5 Implementation Framework Selection: 12
4.4 Experimental Design: 12
4.4.1 Evaluation Metrics: 13
4.4.2 Comparative Evaluation: 13
4.4.3 Baseline Comparison: 13
4.4.4 Experimental Protocol: 13
4.5 Evaluation Methodology 13
4.5.1 Stage 1: Unit Testing 14
4.5.2 Stage 2: Pilot Experiments 14
4.5.3 Stage 3: Systematic Evaluation 14

5 Implementation **14**
5.1 Framework Evaluation and Selection 14
5.1.1 AutoGen (Microsoft) 15
5.1.2 CrewAI 15
5.1.3 LangGraph (Selected Framework) 15
5.2 Framework Comparison Summary 16
5.3 System Architecture 16
5.3.1 Frontend Layer 16
5.3.2 Backend API Layer 17
5.3.3 Agent Core (LangGraph) 17
5.3.4 Database Layer 17
5.4 Implementation Results 18

6 Project Scheduling and Risk Management **18**
6.1 Project Schedule (Gantt Chart) 18
6.1.1 Detailed Weekly Breakdown: 18
6.2 Risk Register and Risk Matrix 19
6.2.1 Implementation Complexity Exceeds Timeline 19
6.2.2 Difficulty in Obtaining Meaningful Evaluation Metrics: 19
6.2.3 Technical dependencies and Library Issues: 20
6.2.4 Scope Creep: 20
6.2.5 Insufficient Experimental Scenario: 20
6.2.6 Non-Deterministic Behaviour Complicates Evaluation: 21
6.2.7 Time Constraints for Full Comparative Evaluation: 21

7 Outlook **21**

References **24**

Appendix **25**
.1 Gantt Chart 25

1 Introduction

1.1 Project Background

Agentic AI is an emerging paradigm in artificial intelligence that refers to autonomous systems capable of pursuing complex goals with limited human oversight (Acharya, Kuppan, and Divya 2025). Unlike traditional rule-based systems, agentic AI systems can dynamically reason about their environment, formulate plans, execute actions, and refine their strategies based on feedback and changing conditions. These systems typically integrate large language models (LLMs) with planning modules, external tool interfaces, and memory mechanisms to achieve goal-directed behaviour (Hughes et al. 2025).

Study planning and coursework management present an interesting domain for exploring agent-based artificial intelligence capabilities. Students must balance multiple assignments, deadlines, and time constraints while adapting to new tasks and changing priorities. Although this context provides a concrete scenario for evaluation, the primary research focus is on the technical capabilities of agentic AI systems rather than on solving real-world student workload problems. The study planning domain serves as a well-defined test-bed for evaluating how autonomous agents handle constraint satisfaction, dynamic replanning, and multi-objective optimisation.

1.2 Project Motivation

Current approaches to task scheduling, such as static to-do lists or simple calendar applications, lack the autonomous reasoning and adaptive planning capabilities that characterise agentic AI systems (Z. Chen et al. 2023). From a research perspective, understanding how agentic architectures can effectively generate and maintain feasible schedules under constraints is an important technical question. This project is motivated by several research gaps:

First, there is limited empirical evaluation of how agentic AI systems perform in realistic constraint-based scheduling scenarios. Although planning algorithms have been extensively studied in classical AI, their integration with modern LLM-based agent frameworks and their performance in dynamic, user-constrained environments remain under-explored.

Second, the relative effectiveness of different agent architectures, such as rule-based planners, LLM-driven planning loops, and hybrid approaches, has not been systematically compared in scheduling contexts. Understanding the trade-offs between explainability, computational efficiency, and solution quality is important to advance agentic AI research.

Third, the robustness of agentic planners to changing requirements, conflicting constraints, and incomplete information represents a fundamental challenge that

warrants empirical investigation. A prototypical study planner provides a controlled environment to explore these issues.

1.3 Research Questions and Hypothesis

The project addresses the following research questions:

1. How effectively can a prototypical agentic AI study planner generate feasible weekly schedules under typical student constraints (deadlines, daily hour limits, task dependencies)?

2. How does an agentic AI planner compare to a baseline rule-based scheduler in terms of schedule feasibility, flexibility, and computational efficiency?

3. How do different planning approaches or agent frameworks (e.g., LLM-based planning loops versus constraint-based algorithms) affect the performance, explainability, and robustness of the study planner?

4. To what extent can an agentic planner adaptively reschedule tasks when new assignments are introduced or existing deadlines change?

The following hypotheses are proposed:

1. An agentic AI planner will generate schedules with higher feasibility rates (fewer constraint violations) compared to a simple rule-based baseline when handling complex, multi-constraint scenarios.

2. LLM-based planning approaches will demonstrate greater flexibility in handling novel constraint combinations but may exhibit lower computational efficiency compared to algorithmic planners.

2 Description of the Project

2.1 Problem Statement

The core research problem is to design and evaluate a prototypical agentic AI system capable of autonomous dynamic task scheduling under multiple constraints. The system must demonstrate key agentic capabilities: perceiving task requirements and constraints, reasoning about feasible schedule configurations, generating and executing planning decisions, and adapting to environmental changes through iterative replanning.

The study planning domain provides a structured use case with well-defined constraints (deadlines, time budgets, task dependencies) that enables controlled evaluation of agentic planning mechanisms. This project is not framed as a real-world application study. Instead, the study planning context serves as a concrete testbed for demonstrating and evaluating core agentic AI technologies. The focus is on understanding how autonomous agents handle constraint-based planning, not on deploying a production system for student use.

2.2 Project Goal and Objectives

1. **Project Goal**: Design, implement, and experimentally evaluate a prototypical agentic AI study planner that demonstrates autonomous scheduling capabilities and provides insight into the effectiveness of different agentic planning approaches

2. **Specific Objectives**:

   (a) specify a comprehensive agent architecture for a study planner that integrates perception, planning, and action modules, capable of generating autonomous schedules and adaptive rescheduling.

   (b) develop a functional prototype of the agentic planner using an appropriate framework(e.g., an LLM-based agent toolkit such as LangChain or AutoGPT, or a custom planning algorithm implementation).

   (c) design and implement experimental scenarios that systematically test the planner's performance under varying constraint configurations, including different numbers of tasks, deadline distributions, and time budget constraints.

   (d) conduct a comparative evaluation of alternative planning strategies or agent frameworks, assessing the trade-off on feasibility, efficiency, explainability, and robustness.

   (e) analyse and document the strengths, limitations, and failure modes of the prototypical system, identifying key insights for future agentic AI research.

   (f) produce a comprehensive research report documenting the design, implementation, evaluation, methodology, results, and contributions.

2.3 Project Deliverables and Milestones

1. **Deliverables**

(a) Conceptual Architecture Document: Detailed specification of the agent architecture, including component descriptions, data flows, and planning mechanisms.

(b) Implementation of the prototype: Functional code base that implements the agentic planner with core scheduling capabilities.

(c) Experimental Design and Results: Results of the systematic evaluation that include performance metrics, comparative analyses, and identified limitations.

(d) Final Research Report: Comprehensive documentation of the project, including review of the literature, methods, results, analysis, and conclusions.

2.4 Project Roadmap

The project will proceed through six main phases: Literature review and requirements, architecture design, prototype implementation , experimental design, and pilot testing. main experiments and analysis, and report writing. A detailed project schedule with weekly breakdown is provided in Section 6

3 Literature Review

3.1 Agentic AI and Autonomous Agents

The field has seen significant development in recent years, with various definitions and conceptualisations emerging. Recent research by Sapkota, Roumeliotis, and Karkee (2025) and Schneider (2025) provides detailed conceptual frameworks distinguishing AI agents from agentic AI and explores the transition from generative AI to more autonomous agentic systems. Bandi et al. (2025) offer a comprehensive review of agentic AI, examining definitions, frameworks, architectures, applications, evaluation metrics, and challenges. Their work identifies key characteristics of agentic systems: autonomous decision-making, tool use, iterative reasoning, and adaptive behaviour. Pati (2025) provide a broad survey of agentic AI technologies, applications, and social implications, emphasising the multi-faceted nature of these systems. A key characteristic of agentic AI is iterative reasoning. Agents can reflect on their actions, receive feedback, and refine their plans (L. Wang et al. 2024). This capability is relevant for scheduling problems, where initial plans may need adjustment based on new information or restrictions. However, current agentic systems often struggle to maintain consistency in planning iterations and handle conflicting information (Murugesan 2025) The literature reveals a gap in empirical evaluation

of agentic AI systems for structured planning problems. While many frameworks exist(e.g. AutoGPT, LangChain agents, ReAct-style systems), their performance in constraint-based scheduling scenarios has not been systematically examined. (Brohi, Jhanjhi, Pillai, et al. 2025) mapped the research landscape of agentic AI and large language models, identifying applications and future directions, but noted limited evaluation in constrained planning domains. This project addresses this gap by providing empirical evaluation in a controlled scheduling domain.

3.2 Planning and Scheduling Algorithms for Intelligent Agents

Classical AI planning algorithms, including STRIPS-style planners, hierarchical task network (HTN) planners, and constraint-based approaches, have been extensively studied in the automated planning literature (Geffner and Bonet 2013; Ghallab, Nau, and Traverso 2016). These methods are effective in generating feasible plans under well-defined constraints, but typically assume a fully specified, static, and deterministic environment, which limits their robustness in dynamic or partially observable settings (Novák and Jamroga 2011; Bidoux, Pignon, and Bénaben 2017).

Modern agentic systems increasingly use LLM-based planning, where language models generate action sequences through iterative prompting (Sun et al. 2023). ReAct frameworks (Reasoning and Acting) combine reasoning traces with tool use, allowing agents to plan and execute interleaved (Yao et al. 2022). However, LLM-based planning can be computationally expensive, non-deterministic, and difficult to verify accuracy (Aghzal et al. 2025)(Katz et al. 2024)(C. P. Lee et al. 2025). This presents challenges for reliable scheduling applications. Self-evolving AI agents represent an advanced category of agentic systems that can adapt and improve their planning capabilities over time. (Fang et al. 2025) surveyed self-evolving agents, examining how these systems can modify their architectures and strategies based on experience. Although promising, these approaches introduce additional complexity and evaluation challenges. Hybrid approaches that combine algorithmic planning with LLM-based reasoning show promise but are still under exploration. For scheduling problems specifically, there is limited research that systematically compares the effectiveness of different planning paradigms within agentic architectures. This project contributes by empirically evaluating alternative planning approaches in a concrete scheduling context.

3.3 LLM-Based and Tool-Using Agent Frameworks

Frameworks such as LangChain, AutoGPT, and BabyAGI provide infrastructure to build LLM-based agents with planning capabilities (Chase et al. 2022). These frameworks typically implement planning loops where agents:

1. observe the current state
2. reason about goals and constraints
3. execute actions using tools
4. update state based on outcomes

A limitation of current LLM-based planning is the absence of rigorous formal constraint handling. Although LLMs can generate plausible plans, they can breach hard constraints or yield infeasible schedules. Exploring ways to integrate formal constraint checking with LLM-based reasoning therefore represents an important research direction. (Aghzal et al. 2025)

The explainability of LLM-based planning also remains a challenge. It is often difficult for users to understand why an agent selected a particular plan or scheduling decision when the underlying reasoning is distributed across high-dimensional neural network activations rather than explicit symbolic structures. Recent work on the governance of agentic AI systems stresses the importance of the legibility of agent activity, including making agents' actions and internal reasoning observable to users and recognising the limitations of existing transparency mechanisms (Shavit et al. 2023). Based on these concerns, this project will investigate methods to improve the transparency and auditability of LLM-based planning decisions.

3.4 Intelligent Scheduling and Study Planning Systems

Existing intelligent scheduling systems, such as AI-powered calendar applications, typically use rule-based or machine learning approaches for optimisation (Z. Chen et al. 2023). However, these systems rarely exhibit true agentic autonomy. They typically require explicit user input for each decision rather than autonomously generating and maintaining schedules.

Research on intelligent tutoring systems and educational planning has explored automated schedule generation, but these systems are typically not framed as agentic AI research (Koedinger et al. 2010). The gap between educational scheduling research and agentic AI research provides an opportunity for this project to contribute new insights.

3.4.1 Critical Gaps Identified

1. Limited empirical evaluation of agentic planners in realistic constraints-based scheduling scenarios with systematic performance metrics.
2. Lack of comparative studies between rule-based, LLM-based, and hybrid planning approaches within agentic architectures.

3. Insufficient understanding of how agentic systems handle conflicting constraints and dynamic replacement in scheduling contexts.

4. Absence of frameworks that combine the flexibility of LLM-based reasoning with the reliability of formal constraint satisfaction.

This project addresses these gaps by providing systematic empirical evaluation and comparative analysis of agentic planning approaches in a controlled scheduling domain. The study planning context offers a structured environment for this evaluation.

4 Project Methods

4.1 Project Type

This project is framed as a design-and-implementation study of a prototypical agentic AI system, combined with experimental evaluation and comparison of core agentic technologies or frameworks. The project does not adopt a focus on real-world applications as its primary classification. While the study planning domain provides a concrete use case, the research emphasis is on understanding and evaluating agentic AI capabilities rather than deploying a production system for real-world student use.

The project involves both system design and implementation and the experimental comparison of alternative planning approaches or agent frameworks. This provides contributions to agentic AI research through the development and evaluation of a prototypical system.

4.2 Requirements Elicitation and Definition

Requirements will be defined through scenario-based specification and analysis of agentic planning needs:

4.2.1 Scenario Definitions:

1. Weekly planning scenarios with 5-15 tasks

2. Task attributes: deadline, estimated duration, priority, dependencies

3. Constraints: maximum daily study hours (e.g., 8 hours), minimum break requirements, preferred time slots

4. Dynamic scenarios: introduction of new tasks mid-week, deadline changes, task cancellation

4.2.2 Functional Requirements:

1. The agent must autonomously generate a feasible weekly schedule with respect to all hard constraints (deadlines, daily hour limits).
2. The agent must be able to reschedule existing tasks when new tasks are introduced or constraints change.
3. The agent must handle task dependencies (e.g., task B cannot start until task A is completed).
4. The agent must provide explanations for scheduling decisions (e.g., why a task was scheduled at a particular time).

4.2.3 Non-Functional Requirements:

1. The scheduling process should be completed in a reasonable time (less than five minutes for typical scenarios).
2. The system should be transparent about constraint violations or infeasible scenarios
3. The architecture should be modular to allow for comparison of different planning modules.

4.3 Design: Agent Architecture

The proposed agent architecture consists of four main components:

4.3.1 Perception Module:

1. Input interface to receive task specifications, deadlines, and constraints.
2. State representation: current schedule, remaining tasks, constraint status.
3. Change detection: identifies when new tasks are added or constraints modified.

4.3.2 Internal Representation:

1. Data Structures for tasks (attributes, dependencies, status)
2. Constraint model (deadlines, time budgets, preferences).
3. Schedule state (allocated time slots, assigned tasks).

4.3.3 Planning Module (core Component:

1. Option A (LLM-bsed): Iterative planning loop using LLM prompts to generate schedule steps, with constraint checking at each iteration.
2. Option B (Algorithmic): Constraint-based scheduling algorithm (e.g., greedy heuristic with backtracking, CSP solver).
3. Option C (Hybrid): LLM generates the high-level plan structure, the algorithmic module refines, and validates the constraints. The planning module will be designed to be swappable, allowing for a comparative evaluation of different planning approaches.

4.3.4 Action/Output Module:

1. Schedule generation: produces time-allocated task assignments.
2. Explanation generation: provides reasoning traces for scheduling decisions.
3. Update mechanism: modifies existing schedule when replanning is triggered.

4.3.5 Implementation Framework Selection:

The prototype will be developed in Python, potentially using:

1. LangChain or similar framework for LLM-based agent components
2. Custom Python implementation for algorithmic planning modules
3. Constraint satisfaction libraries (e.g., python-constraint) if needed

4.4 Experimental Design:

1. Scenario Set 1 (Simple): 5-7 tasks, no dependencies, uniform deadlines
2. Scenario Set 2 (Moderate): 10-12 tasks, some dependencies, varied deadlines
3. Scenario Set 3 (Complex): 15+ tasks, multiple dependencies, tight deadlines, conflicting constraints
4. Dynamic Scenarios: Introduction of 2-3 new tasks to existing schedules

4.4.1 Evaluation Metrics:

1. Feasibility Rate: Percentage of generated schedules that satisfy all hard constraints
2. Constraint Violations: Count and severity of violations (deadline misses, hour limit exceedances)
3. Computational Efficiency: Time to generate schedule (Seconds)
4. Robustness: Ability to successfully reschedule when new tasks are introduced (success rate)
5. Schedule Quality: Heuristic measures such as deadline buffer time, distribution of work hours

4.4.2 Comparative Evaluation:

If multiple planning approaches are implemented:

1. Direct comparison of feasibility rates, efficiency, and robustness
2. Qualitative analysis of explainability and failure modes
3. Statistical analysis of performance differences across scenario sets

4.4.3 Baseline Comparison:

1. Baseline 1: Simple rule-based scheduler (earliest deadline first, fill available hours)
2. Baseline 2: Manual schedule (designed by humans for comparison scenarios)

4.4.4 Experimental Protocol:

1. Each scenario will be evaluated 10 times to account for non-determinism (if applicable)
2. The results will be aggregated and statistically analysed
3. Failure cases will be qualitatively documented and analysed.

4.5 Evaluation Methodology

The evaluation will proceed through three stages:

4.5.1 Stage 1: Unit Testing

1. Individual component testing (constraint checking, task representation, etc.)
2. Validation of core functionality before integration

4.5.2 Stage 2: Pilot Experiments

1. Small-scale scenario testing (5-10 scenarios)
2. Refinement of metrics and experimental protocol
3. Identification of implementation issues

4.5.3 Stage 3: Systematic Evaluation

1. Execution of the full scenario set
2. Comparative analysis (if multiple approaches are implemented)
3. Statistical analysis of results
4. Qualitative analysis of failure modes and edge cases

Success criteria include:

1. The prototype successfully generates feasible schedules for at least 80% of simple and moderate scenarios.
2. Clearly demonstrates advantages or trade-offs compared to the baseline approaches.
3. Provides insights into agentic planning capabilities and limitations.

5 Implementation

This section describes the implementation of the Agentic AI Academic Task Manager prototype, including the framework evaluation process, technology selection rationale, and system architecture.

5.1 Framework Evaluation and Selection

Three leading agentic AI frameworks were evaluated to determine the most suitable foundation for the academic task management system: AutoGen (Q. Wu et al. 2023), CrewAI (CrewAI 2024), and LangGraph (LangChain AI 2024).

5.1.1 AutoGen (Microsoft)

AutoGen provides multi-agent conversation capabilities, enabling collaborative AI tasks through agent-to-agent communication. The framework was tested with a minimal implementation comprising an AssistantAgent and UserProxyAgent. While AutoGen excels at conversational AI workflows, it demonstrated several limitations for our use case:

- Heavy dependency on external LLM APIs for all operations
- Complex configuration requirements for simple, structured workflows
- Architecture optimised for conversations rather than state management

Verdict: Not selected—designed primarily for multi-agent conversations rather than structured CRUD operations.

5.1.2 CrewAI

CrewAI implements a role-based agent design where agents are assigned specific roles, goals, and backstories. The framework provides good abstraction for team-like AI workflows. Testing revealed:

- Requires LLM API calls for every operation, increasing costs
- Role-based paradigm does not align well with task management workflows
- Steeper learning curve for straightforward database operations

Verdict: Not selected—role-based model does not fit CRUD-centric task management.

5.1.3 LangGraph (Selected Framework)

LangGraph (LangChain AI 2024), developed by LangChain AI, provides a graph-based approach to building stateful, multi-actor applications. The framework was evaluated with a state-driven workflow implementation using TypedDict for state management and conditional edges for routing. Key advantages identified:

- Explicit state management with TypedDict enables type-safe operations
- Graph-based workflow design provides clear, deterministic routing
- Conditional edges support dynamic decision-making without LLM overhead
- Direct database integration is straightforward

- Core operations function without requiring LLM API calls

Verdict: Selected—best fit for structured, state-driven task management with cost efficiency.

5.2 Framework Comparison Summary

Table 1 summarises the comparative evaluation of the three frameworks.

Criteria	**AutoGen**	**CrewAI**	**LangGraph**
State Management	Limited	Limited	Excellent
LLM Dependency	High	High	Optional
Workflow Control	Conversational	Role-based	Graph-based
Database Integration	Complex	Complex	Simple
Cost Efficiency	Low	Low	High
CRUD Suitability	Poor	Poor	Excellent

Table 1: Framework comparison for academic task management

5.3 System Architecture

The implemented system follows a three-tier architecture comprising a React frontend (Meta Platforms 2013), Flask backend (Ronacher 2010), and PostgreSQL database (The PostgreSQL Global Development Group 1996), with LangGraph serving as the agent core.

5.3.1 Frontend Layer

The frontend is implemented using React 18.2 with functional components and hooks. Key features include:

- Modern UI with CSS gradients, card-based layouts, and responsive design
- User management interface with creation, selection, and listing
- Task CRUD operations with status, priority, and deadline management
- Email reminder triggers for upcoming deadlines
- Environment-based API URL configuration for deployment flexibility

5.3.2 Backend API Layer

The backend is implemented using Flask with the following characteristics:

- RESTful API endpoints: `/api/users`, `/api/tasks`, `/api/reminders`
- CORS support for cross-origin requests
- JSON response formatting with structured data parsing
- Environment-based configuration via python-dotenv
- Integration with LangGraph agent workflow for all operations

5.3.3 Agent Core (LangGraph)

The LangGraph agent implements a StateGraph with the following components:

- **State Definition:** TypedDict-based AgentState containing message history
- **Router Function:** Analyses command prefixes to determine routing
- **Node Functions:** Dedicated handlers for each operation type:
  - help, show_tasks, add_task, update_task, delete_task
  - create_user, get_user, update_user, delete_user, list_users
  - reminders, export_tasks, import_tasks
- **Conditional Edges:** Graph-based routing from START through router to specific nodes, terminating at END

5.3.4 Database Layer

PostgreSQL serves as the persistent storage layer with the following schema:

- **user_profiles:** id (SERIAL PRIMARY KEY), name, email (UNIQUE), academic_program
- **tasks:** id (SERIAL PRIMARY KEY), user_id (FOREIGN KEY), title, description, deadline, priority, status, created_at
- Foreign key constraints ensure referential integrity with CASCADE deletion

5.4 Implementation Results

The implemented prototype successfully demonstrates:

1. **User Management:** Full CRUD operations for academic users with email and program tracking
2. **Task Management:** Complete task lifecycle management with filtering, sorting, and status tracking
3. **Reminder System:** SMTP-based email notifications for upcoming deadlines
4. **Data Portability:** CSV export/import functionality for backup and migration
5. **Deployment Readiness:** Configuration for both local development and cloud deployment (Render, Vercel, Heroku)

The LangGraph-based architecture proved effective for this use case, providing deterministic routing, efficient state management, and straightforward database integration without the overhead of LLM API calls for routine operations.

6 Project Scheduling and Risk Management

6.1 Project Schedule (Gantt Chart)

The project timeline spans 14 weeks and is structured into six main phases. The complete Gantt chart is presented in the Appendix .1 (Table 2

6.1.1 Detailed Weekly Breakdown:

1. Weeks 1-2: Review of the literature (agentic AI, planning algorithms, frameworks)
2. Week 3: Recruiting of requirements and definition of scenarios
3. Week 4: Architecture specification and framework selection
4. Weeks 5-6: Detailed design of components and interfaces
5. Weeks 7-8: Core implementation (data structures, basic planning)
6. Week 9: Integration and initial testing
7. Week 10: Experimental framework and pilot testing

8. Week 11: Execution of the main experiment
9. Week 12: Data analysis and interpretation
10. Weeks 13-14: Report writing, results documentation, finalisation

6.2 Risk Register and Risk Matrix

6.2.1 Implementation Complexity Exceeds Timeline

1. Description: The agent framework or planning approach chosen is more complex to implement than anticipated, delaying prototype development.
2. Probability: Medium
3. Impact: High
4. Mitigation:
   (a) Start with a simpler planning approach; iterate to complexity
   (b) Allocate buffer time according to the schedule (Weeks 7-9)
   (c) Have a fallback to simpler algorithmic approach if LLM-based implementation proves too challenging

6.2.2 Difficulty in Obtaining Meaningful Evaluation Metrics:

1. Description: Defining and measuring "good schedules" may be ambiguous, making comparative evaluation difficult.
2. Probability: Medium
3. Impact: Medium
4. Mitigation:
   (a) Define clear objective metrics early(feasibility, constraint violations).
   (b) Pilot test metrics in week 10 to validate usefulness
   (c) Include both quantitative and qualitative evaluation methods

6.2.3 Technical dependencies and Library Issues:

1. Description: Required libraries (e.g., LLM APIs, constraint solvers) may have compatibility issues, API changes, or access limitations.
2. Probability: Low
3. Impact: High
4. Mitigation:
   (a) Test dependencies early (weeks 4-5).
   (b) Identify alternative libraries/frameworks.
   (c) Use local/offline solutions where possible to avoid API dependency

6.2.4 Scope Creep:

1. Description: Temptation to add features (e.g., user interface, calendar integration) beyond core agentic planning research.
2. Probability: Medium
3. Impact: Medium
4. Mitigation:
   (a) Observe strict requirements and objectives.
   (b) Document "future work" items separately.
   (c) Regular scope review against project goals

6.2.5 Insufficient Experimental Scenario:

1. Description: The limited diversity of scenarios may not provide robust evaluation or meaningful insight.
2. Probability: Low
3. Impact: Medium
4. Mitigation:
   (a) Define Scenario sets early with clear rationale.
   (b) Include edge cases (very tight deadlines, many dependencies).
   (c) Pilot testing will reveal whether scenarios are sufficient

6.2.6 Non-Deterministic Behaviour Complicates Evaluation:

1. Description: LLM-based planning may produce inconsistent results, making statistical analysis difficult.
2. Probability: Medium
3. Impact: Low
4. Mitigation:
   (a) Run multiple trials per scenario (10 repetitions).
   (b) Use appropriate statistical methods for variability.
   (c) Consider this a feature to analyse (robustness evaluation)

6.2.7 Time Constraints for Full Comparative Evaluation:

1. Description: Implementing multiple planning approaches for comparative evaluation may exceed the available time.
2. Probability: Medium
3. Impact: Medium
4. Mitigation:
   (a) Prioritise a single approach for the core prototype.
   (b) Comparative evaluation can be limited to 2 approaches if time-constrained
   (c) Document limitations and future work for an expanded comparison.

In conclusion, the critical risks with high impact and medium or high likelihood are the complexity of implementation and technical dependencies.

7 Outlook

This research project contributes to understanding how agentic AI systems can handle constraint-based structured planning problems. By designing and evaluating a prototypical study planner, the project provides empirical insights into the feasibility, strengths, and limitations of autonomous scheduling agents.

The anticipated contributions are:

1. Technical insights into how different agentic planning approaches (LLM-based, algorithmic, hybrid) perform in realistic constraint-based scenarios, including trade-offs in feasibility, efficiency, and explainability.

2. Methodological contributions through the development of evaluation frameworks and metrics for assessing agentic planning systems in scheduling domains, which can be applied to other planning problems.

3. Identification of limitations through clear documentation of failure modes, edge cases, and scenarios where agentic planners struggle, informing future research directions.

4. Comparative analysis: if multiple approaches are evaluated, information on when different planning paradigms are most appropriate.

Future directions for extending this work include

1. Integration with real systems: future work could integrate the planner with actual calendar applications or learning management systems, though this would shift toward real-world application research.

2. Multi-agent scenarios: exploring how multiple agentic planners could coordinate when managing schedules for groups of students or collaborative projects.

3. Advanced planning techniques: incorporating more sophisticated planning algorithms (e.g., temporal planning, resource-constrained project scheduling) or learning-based approaches that improve over time.

4. Scalability evaluation: testing the planner's performance with larger numbers of tasks (50+ tasks) or longer planning horizons (semester-level planning).

5. User preference learning: extending the agent to learn and adapt to individual user preferences and working patterns through interaction.

SLEP considerations:

Although detailed societal, legal, ethical and professional (SLEP) analysis is outside the scope of this cw2 proposal, future work should address the following:

1. Societal: impact of automated planning on student autonomy and learning skills; accessibility considerations for diverse student needs.

2. Legal: data privacy concerns if the system processes personal academic information; compliance with educational data protection regulations.

3. Ethical: bias in scheduling recommendations; transparency and user control over automated decisions; avoiding over-reliance on AI planning.

4. Professional: integration with academic integrity policies; appropriate use of AI tools in educational contexts.

These SLEP considerations would be essential for any real-world deployment, but are noted here as important future research directions rather than the current project scope.

References

Acharya, Deepak Bhaskar, Karthigeyan Kuppan, and B. Divya (2025). "Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey". In: *IEEE Access* 13, pp. 18912–18936. DOI: 10.1109/ACCESS.2025.3532853.

Aghzal, Mohamed et al. (2025). "A survey on large language models for automated planning". In: *arXiv preprint arXiv:2502.12435*.

Bandi, Ajay et al. (2025). "The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges". In: *Future Internet* 17.9, p. 404.

Bidoux, Loïc, Jean-Paul Pignon, and Frédérick Bénaben (2017). "On the use of automated planning for crisis management." In: *ISCRAM*.

Brohi, Sarfraz, NZ Jhanjhi, Thulasyammal Ramiah Pillai, et al. (2025). "A Research Landscape of Agentic AI and Large Language Models: Applications, Challenges and Future Directions". In: *Algorithms* 18.8, p. 499.

Chase, Harrison et al. (2022). "LangChain: Building applications with LLMs through composability". In: *LangChain: Building applications with LLMs through composability, GitHub*.

Chen, Zhiyuan et al. (2023). "A survey on large language model based autonomous agents". In: *arXiv*.

CrewAI (2024). *CrewAI: Framework for orchestrating role-playing, autonomous AI agents*. https://github.com/joaomdmoura/crewAI. Accessed: 2026-02-28.

Fang, Jinyuan et al. (2025). "A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems". In: *arXiv preprint arXiv:2508.07407*.

Geffner, Hector and Blai Bonet (2013). *A concise introduction to models and methods for automated planning*. Morgan & Claypool Publishers.

Ghallab, Malik, Dana Nau, and Paolo Traverso (2016). *Automated planning and acting*. Cambridge University Press.

Hughes, Laurie et al. (2025). "AI agents and agentic systems: A multi-expert analysis". In: *Journal of Computer Information Systems*, pp. 1–29.

Katz, Michael et al. (2024). "Thought of search: Planning with language models through the lens of efficiency". In: *Advances in Neural Information Processing Systems* 37, pp. 138491–138568.

Koedinger, Kenneth R et al. (2010). "A data repository for the EDM community: The PSLC DataShop". In: *Handbook of educational data mining* 43, pp. 43–56.

LangChain AI (2024). *LangGraph: Build stateful, multi-actor applications with LLMs*. https://github.com/langchain-ai/langgraph. Accessed: 2026-02-28.

Lee, Christine P et al. (2025). "Veriplan: Integrating formal verification and llms into end-user planning". In: *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–19.

Meta Platforms (2013). *React: A JavaScript library for building user interfaces.* `https://reactjs.org/`. Accessed: 2026-02-28.

Murugesan, San (2025). "The rise of agentic AI: implications, concerns, and the path forward". In: *IEEE Intelligent Systems* 40.2, pp. 8–14.

Novák, Peter and Wojciech Jamroga (2011). "Agents, actions and goals in dynamic environments". In: *IJCAI*, pp. 313–318.

Pati, Ashis Kumar (2025). "Agentic AI: A Comprehensive Survey of Technologies, Applications, and Societal Implications". In: *IEEE Access.*

Ronacher, Armin (2010). *Flask: A Python Microframework.* `https://flask.palletsprojects.com/`. Accessed: 2026-02-28.

Sapkota, Ranjan, Konstantinos I Roumeliotis, and Manoj Karkee (2025). "Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges". In: *arXiv preprint arXiv:2505.10468.*

Schneider, Johannes (2025). "Generative to agentic ai: Survey, conceptualization, and challenges". In: *arXiv preprint arXiv:2504.18875.*

Shavit, Y et al. (2023). *Practices for governing agentic AI systems. OpenAI.*

Sun, Haotian et al. (2023). "Adaplanner: Adaptive planning from feedback with language models". In: *Advances in neural information processing systems* 36, pp. 58202–58245.

The PostgreSQL Global Development Group (1996). *PostgreSQL: The World's Most Advanced Open Source Relational Database.* `https://www.postgresql.org/`. Accessed: 2026-02-28.

Wang, Lei et al. (2024). "A survey on large language model based autonomous agents". In: *Frontiers of Computer Science* 18.6, p. 186345.

Wu, Qingyun et al. (2023). *AutoGen: Enabling next-generation LLM applications via multi-agent conversation.* `https://github.com/microsoft/autogen`. Accessed: 2026-02-28.

Yao, Shunyu et al. (2022). "React: Synergizing reasoning and acting in language models". In: *The eleventh international conference on learning representations.*

Appendix

.1 Gantt Chart

Phase	**Weeks**													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Literature Review & Requirements														
Architecture Design														
Implementation														
Experimental Design														
Main Experiments														
Report Writing														

Table 2: Gantt chart for the 14-week project schedule